THE HEALING COLLECTIVE

Product Requirements Document

Adaptive Self-Healing Intelligence for the E-T Systems Ecosystem

Version 1.0 • February 2026 • CONFIDENTIAL
Author: Josh + Claude
License: AGPL-3.0 (see NeuroGraph LICENSE)

Implementation-grade specification for Claude Code handoff
v1.0: All architectural decisions resolved. Full integration spec. Congregation semantics defined.

Table of Contents

0. Implementation Preamble

CRITICAL: This PRD is the authoritative specification for the Healing Collective module. Claude Code MUST NOT deviate from the architecture, file naming, integration patterns, or conventions described here. When in doubt, follow the patterns established by TrollGuard and The Inference Difference — they are the canonical reference implementations.

0.1 Binding Constraints

The following constraints are non-negotiable. Violating any of them produces an incompatible module that cannot participate in the E-T Systems ecosystem:

- **Vendored file identity.** The module MUST vendor exact copies of ng_lite.py, ng_peer_bridge.py, ng_ecosystem.py, and openclaw_adapter.py from the NeuroGraph repo. These files MUST NOT be modified, wrapped, subclassed, or monkey-patched. They are identical across every E-T Systems module.
- **Singleton pattern.** The OpenClaw hook (healing_collective_hook.py) MUST expose a module-level get_instance() function returning a singleton. The singleton MUST be an OpenClawAdapter subclass. This is the only entry point OpenClaw uses.
- **Module ID.** module_id = "healing_collective" everywhere: et_module.json, ng_ecosystem.init(), hook class, JSONL events, shared learning directory. No variations (no hyphens, no camelCase, no abbreviations).
- **et_module.json v2 schema.** MUST use _schema: "et_module/2.0" with the full ecosystem block

including tier3_upgrade: true, ng_ecosystem_version: "1.0.0", capabilities, and provides fields.

- **Shared learning directory.** All peer bridge events go to ~/.et_modules/shared_learning/healing_collective.jsonl. Per-module state goes to ~/.et_modules/healing_collective/. These paths are NOT configurable — they are dictated by the integration standard.
- **Living changelog.** Every .py file MUST have a changelog block in the header docstring using the standard format: [YYYY-MM-DD] Author — Title. With What/Why/Settings/How subsections.
- **Graceful degradation.** Every tier upgrade attempt, every external import, every I/O operation MUST be wrapped in try/except. A failed Tier 3 probe MUST NOT affect Tier 1 or 2 operation. A missing peer bridge MUST NOT prevent standalone operation.
- **validate() before execute().** Every repair primitive MUST implement validate() and execute() as separate methods. validate() MUST be called before execute() with no exceptions. validate() MUST NOT have side effects.
- **No runtime LLM dependency.** The Collective MUST NOT require any LLM, API key, or network call for its core diagnostic and repair operations. All intelligence comes from the NG-Lite substrate and DVS. The module runs entirely client-side.

0.2 Reference Implementations

Before writing any code, study these files in their respective repos:

File	Repo	What It Shows
trollguard_hook.py	TrollGuard	Canonical OpenClawAdapter subclass: _embed(), _module_on_message(), _module_stats(), get_instance()
ng_ecosystem.py	NeuroGraph (root)	Tier 1→2→3 lifecycle, singleton factory, auto-upgrade polling, record_outcome/get_recommendations
openclaw_adapter.py	NeuroGraph (root)	OpenClawAdapter ABC: on_message/recall/stats contract, event logging to JSONL, hash_embed fallback
et_module.json	TrollGuard	v2 manifest with full ecosystem block: ng_lite, peer_bridge, tier3_upgrade, capabilities, provides
install.sh	TrollGuard	Registration script: creates ~/.et_modules dirs, registers with ET Module Manager
ET_MODULE_INTEGRATION_PRIMER_v2	NeuroGraph (docs)	The full integration standard — the law of the land

0.3 What This Module Is NOT

- **NOT a port of the TypeScript proto.** The proto (healer-collective.ts) was procedural orchestration: spawn workers, assign tasks, run AI diagnosis, vote, execute. The Healing Collective is a learning system. Its intelligence comes from the NG-Lite substrate, not from scripted workflows.
- **NOT a replacement for Immunis.** Immunis detects threats and triages them. The Collective repairs what Immunis identifies. They are complementary. When Immunis is absent, the Collective detects failures on its own.
- **NOT dependent on NeuroGraph proper.** MUST be fully functional at Tier 1 with only ng_lite.py. NeuroGraph adds SNN capabilities but is never required.
- **NOT a runbook engine.** Does not ship with IF-THEN repair rules. Ships with repair primitives and a learning substrate. Which primitive to apply to which failure is learned through experience.
- **NOT a monitoring dashboard.** The Collective is a healing intelligence, not a visualization tool. It exposes stats via _module_stats() for other tools to display, but it does not render UIs.

1. Executive Summary

1.1 Vision

The Healing Collective is a learning self-healing module for the E-T Systems ecosystem. It observes failures across any host system and any connected peer module, learns what repairs work, and applies that knowledge automatically. It does not follow playbooks. It builds institutional repair wisdom through experience.

At Tier 1, it is a standalone self-healing layer for a single host application. At Tier 2, it absorbs failure patterns and repair knowledge from every co-located E-T Systems module. At Tier 3, backed by NeuroGraph Foundation, it predicts failures before they occur and recognizes complex multi-symptom failure modes through hyperedge pattern completion.

Every E-T Systems module is model-agnostic, client-side, and makes any AI system it connects to better, safer, and stronger. The Healing Collective is no exception. It runs locally, learns locally, and keeps all healing intelligence on the host.

1.2 Problem Statement

Current approaches to system healing are brittle. Runbook-based automation can only fix what someone has pre-scripted. Health-check monitors detect symptoms but cannot diagnose root causes. Self-healing platforms require cloud connectivity and centralized orchestration.

The fundamental limitation is that none of these approaches learn. A system that was repaired once does not become easier to repair the second time. Patterns that recur across different subsystems are invisible. The repair crew has no institutional memory.

1.3 Solution Architecture

The Healing Collective integrates two complementary intelligence layers into the standard E-T Systems module architecture:

The NG-Lite Learning Substrate provides the associative intelligence. Failure patterns map to nodes. Repair actions map to targets. Hebbian learning strengthens successful repair pathways and weakens failed ones. Cross-module patterns flow through the peer bridge. At Tier 3, full STDP and hyperedge discovery enable causal reasoning and composite failure recognition.

The Diagnostic Vector Store (DVS) provides the episodic memory. Full stack traces, repair step sequences, system state snapshots, and diagnostic transcripts are stored with embeddings and retrieved via the substrate's learned topology. This is not flat cosine search — the DVS queries are routed through the substrate's synaptic graph, surfacing results that the system has learned are causally relevant, not merely geometrically similar.

2. System Architecture

2.1 Design Principles

Learning over scripting. The Collective does not ship with repair playbooks. It ships with a learning substrate and a set of repair primitives. The topology of what-fixes-what is learned through experience, not programmed.

Hive mind over instances. The Healing Collective's hive mind is its shared learning substrate: every failure pattern, every repair outcome, every diagnostic chain is a synaptic association that strengthens or weakens with experience. The "collective" is the accumulated wisdom, not a pool of workers.

Host-first, ecosystem-second. The primary client is the host application. At Tier 1 with zero peers, the host already benefits from a self-healing layer that learns its failure patterns.

The VDB has a brain. The DVS does not use flat cosine search. The NG-Lite substrate's learned synaptic topology serves as the VDB's search algorithm. At Tier 3, NeuroGraph's prime_and_propagate provides full SNN-augmented recall.

Standard module contract. Vendors the same four files as every E-T Systems module. Registers with the ET Module Manager at install. Follows the OpenClawAdapter pattern. No special treatment.

Validate before acting. Every repair primitive implements validate()/execute(). validate() runs before execute() with no exceptions. This is the Collective's Hippocratic oath.

Three intake channels. Failures arrive via: (1) peer bridge events, (2) host API calls, (3) active health monitoring. All three are always active. The full repair crew sees everything.

2.2 Module Architecture

The Healing Collective is structured as eight layers:

2.2.1 Vendored Ecosystem Layer

The standard E-T Systems integration files, identical across all modules:

File	Source	Purpose
ng_lite.py	NeuroGraph repo root	Tier 1 Hebbian learning substrate
ng_peer_bridge.py	NeuroGraph repo root	Tier 2 peer-to-peer learning bridge via shared_learning directory
ng_ecosystem.py	NeuroGraph repo root	Tier 1→2→3 orchestration singleton with auto-upgrade detection
openclaw_adapter.py	NeuroGraph repo root	OpenClaw skill hook ABC: on_message/recall/stats

> **CONSTRAINT:** These four files are vendored verbatim. They MUST NOT be modified. Any Healing Collective-specific logic goes in healing_collective_hook.py and the core/ subdirectory.

2.2.2 Three-Channel Failure Intake

The Collective receives failure signals through three independent channels. All three are always active. No channel depends on any other.

Channel 1: Peer Bridge Events (passive). The NGPeerBridge reads JSONL event files from ~/.et_modules/shared_learning/. When a peer module writes a failure event, the Collective absorbs it during its sync cycle. Standard Tier 2 mechanism — no custom protocol. The Collective just reads what peers write.

Channel 2: Host API (explicit). The hook exposes report_failure(description: str, metadata: dict = None) -> str. The host calls this when it detects a problem. Returns a tracking ID (UUID). This is the primary intake for host-system failures not visible in the peer bridge (API gateway timeouts, database connection pool exhaustion, user-reported errors).

Channel 3: Active Health Monitor (proactive). A background thread periodically reads peer module NG-Lite state files from ~/.et_modules/<peer_id>/ng_lite_state.json and analyzes for anomalies: weight divergence, firing rate collapse, novelty saturation, and state file corruption. Detects problems the affected module may not be aware of.

Health Monitor Implementation Notes:

- **Thread model:** A single daemon thread (threading.Thread, daemon=True) launched at hook initialization. NOT multiprocessing. NOT asyncio. This keeps the Collective in-process with the host, matching TrollGuard's threading model.
- **Peer discovery:** The health monitor discovers peers by listing directories in ~/.et_modules/ and checking for ng_lite_state.json files. It does NOT use the NGPeerBridge peer list (which only tracks learning events). This means the monitor can detect unhealthy modules even before the bridge has synced.
- **Rate limiting:** The monitor reads each peer's state file at most once per health_monitor.interval_seconds (default 120). It caches the previous state for delta comparison (weight divergence calculation).

Parameter	Default	Description
health_monitor.enabled	true	Enable/disable active monitoring
health_monitor.interval_seconds	120	Seconds between health check cycles
health_monitor.weight_divergence_threshold	2.0	Max acceptable weight delta between checks (L2 norm)
health_monitor.min_firing_rate	0.001	Below this, flag module as learning-stalled
health_monitor.novelty_saturation_threshold	0.95	Avg novelty above this flags loss of trained patterns

2.2.3 Diagnostic Vector Store (DVS)

A compact, NG-Lite-augmented vector database for healing operations. Stores failure signatures, repair transcripts, system state snapshots, and diagnostic context with semantic embeddings.

Key design: The DVS routes ALL search through the NG-Lite substrate's learned topology. When the Collective queries for similar failures, results are ranked by the substrate's learned associations — what the system has empirically connected to the query pattern — not merely by cosine similarity.

DVS Entry Schema:

Field	Type	Description
entry_id	str (UUID4)	Unique identifier
entry_type	enum str	FAILURE_SIGNATURE | REPAIR_RECORD | STATE_SNAPSHOT | DIAGNOSTIC_LOG | COMPRESSED_PATTERN
timestamp	float	Unix epoch of creation
source_module	str	Module that generated this entry (module_id or "host")
embedding	list[float]	384-dim (Tier 1/2) or 768-dim (Tier 3) semantic vector
content	dict	Type-specific payload (stack trace, repair steps, state data, etc.)
ng_node_id	int | None	NG-Lite node this entry is linked to (populated after first substrate interaction)
repair_outcome	str | None	"success" | "partial" | "failed" | None (failure signatures have no outcome yet)
confidence	float	Substrate confidence at time of creation [0.0, 1.0]
compression_sources	list[str] | None	Entry IDs merged into this compressed entry (COMPRESSED_PATTERN only)
ttl_accesses	int	Access counter for LRU eviction (incremented on every DVS query that returns this entry)

DVS Persistence: Primary format is msgpack. File: ~/.et_modules/healing_collective/dvs.msgpack. A JSON export utility (dvs_export_json()) is provided for debugging. The msgpack file is the source of truth. JSON is never loaded at startup.

DVS Size: Configurable via dvs_max_entries (default 10000). LRU eviction based on ttl_accesses. Pattern compression reduces effective storage needs (see §2.2.7).

2.2.4 Repair Primitive Layer

The Collective's action vocabulary. Each primitive is a small, safe operation the Collective can learn to apply.

Primitive Interface (ABC):

```
class RepairPrimitive(ABC):
  @abstractmethod
  def validate(self, context: dict) -> ValidationResult: ...
  @abstractmethod
  def execute(self, context: dict) -> ExecutionResult: ...
```

ValidationResult fields: passed: bool, reason: str, preconditions: dict

ExecutionResult fields: status: "success" | "partial" | "failed", detail: str, rollback_info: dict | None, duration_ms: float

ENFORCEMENT: validate() is called by the Diagnosis Engine, not by individual primitives. The Diagnosis Engine MUST NOT call execute() without a preceding validate() that returned passed=True. This is enforced in code, not by convention.

Initial Repair Primitives:

Primitive	Scope	validate() Check	execute() Action	Reversible
config_adjust	Host	Value within safe bounds per key	Modify config file/env var	Yes
process_restart	Host	Process exists, restartable, not in cooldown	SIGTERM → wait → verify restart	No (idempotent)
cache_clear	Host	Cache dir/key exists	Remove cached data	No (safe)
retry_with_backoff	Host	Target endpoint reachable	Retry with exponential backoff	N/A
ng_lite_rebalance	Peer	Fork state → apply → verify healthier	Homeostatic weight scaling on peer NG-Lite	Yes (checkpoint)
checkpoint_restore	Peer	Checkpoint file exists and parseable	Replace peer NG-Lite state with checkpoint	Yes (backup first)
connection_pool_reset	Host	Pool handle accessible	Drain and reinitialize pool	No (safe)
log_and_recommend	Any	Always passes	Log diagnostic chain, emit recommendation event	N/A

Custom primitives: Host applications and modules register custom primitives via register_primitive(name, instance). Each MUST implement the RepairPrimitive ABC. The substrate learns their effectiveness through outcome feedback.

Sandbox Capability (self-contained):

The Collective owns its own sandbox. This is NOT delegated to Immunis.

- **Graph-level repairs:** Fork target NG-Lite state into temp copy → apply repair to copy → run health checks (weight distribution, novelty accuracy, pattern fidelity) → promote to live state only if copy is healthier than original.
- **Host-level repairs:** validate() dry-run serves as sandbox. For config_adjust: store original value before execute(), revert if post-repair health checks fail within revert_window_seconds (default 60).

2.2.5 Diagnosis Engine

The intelligence layer connecting failure observation to repair action. Leverages the NG-Lite substrate's learned associations, not rule-based scoring.

Seven-Step Diagnosis Pipeline:

1. **Observe.** A failure event arrives from any intake channel. The event is embedded and stored in the DVS as a FAILURE_SIGNATURE entry.
2. **Recognize.** The embedding activates the NG-Lite substrate. Novelty detection determines familiarity. Low novelty = known pattern, substrate lights up repair pathways. High novelty = new failure, Collective enters cautious mode (lower confidence ceiling).
3. **Recall.** DVS is queried through the substrate's activated topology, surfacing repair records, diagnostic logs, and state snapshots associated with similar past failures. At Tier 3: NeuroGraph's prime_and_propagate for SNN-augmented recall with causal chain discovery.
4. **Propose.** Based on recalled repair knowledge and current system context, the Engine proposes

a repair action (primitive + parameters). Confidence derives from synaptic weights: a repair that has succeeded many times has high confidence. No strong association = low confidence.

5. **Validate.** The proposed repair's validate() is called. Failure → downgrade to recommendation or discard. Pass but low confidence → log as recommendation. validate() failure reason is stored in DVS for learning.
6. **Execute.** If confidence ≥ auto-execute threshold AND validate() passed: call execute(). Outcome (success/partial/failed) is recorded and fed back via record_outcome(), strengthening or weakening the failure-to-repair pathway.
7. **Learn.** The complete diagnostic chain (failure → diagnosis → repair → outcome) is stored in DVS and shared via peer bridge. Every connected module and future Collective instance benefits. Substrate topology is permanently updated.

Confidence Thresholds:

Threshold	Default	Action
Auto-execute	0.70	Execute repair automatically (validate() must still pass)
Recommend	0.40	Log recommendation with full diagnostic chain; emit event for human review
Silent log	< 0.40	Log to DVS only. No action, no recommendation. Substrate still learns from observation.
Host repair premium	+0.15	Host-system repairs require confidence 0.15 higher than peer repairs (i.e., 0.85 for auto-execute)

2.2.6 Congregation Semantics

RESOLVED: The congregation is NOT a multi-agent voting protocol. It is a substrate convergence mechanism for complex failures.

The proto's congregation spawned instances, proposed repairs, voted, and reached quorum. That was procedural orchestration. In the NG module, the "congregation" emerges from the substrate's natural behavior when processing complex failures:

Simple failures: One strong pathway activates. Diagnosis Engine proposes and executes. No congregation needed. This is the common case.

Complex failures (congregation trigger): When a failure activates multiple repair associations with similar confidence (within congregation.confidence_spread, default 0.15), the substrate is uncertain. The Collective does NOT pick the highest and run with it.

Tier 1/2 Deliberation Mode:

- Run the Recall step for each competing pathway, enriching diagnosis with DVS context from each.
- Score proposals against current system state: which repair's preconditions best match reality? Which has succeeded most recently? Which has the shortest repair chain?
- Winning proposal proceeds to Validate/Execute. Losers are logged for learning.
- Deliberation adds latency (bounded by congregation.timeout_seconds) but prevents acting on ambiguous signals.

Tier 3 SNN Convergence:

- Competing pathways inhibit each other through lateral inhibition (existing NeuroGraph SNN dynamics).

- The pathway accumulating the most activation “wins” through network convergence — a single propagation step.
- Hyperedge membership provides additional signal: if the failure matches a composite signature, the associated repair pathway gets a pattern-completion boost that resolves ambiguity.
- Mathematically equivalent to a voting protocol but runs in one propagation step, not a multi-round consensus.

Parameter	Default	Description
congregation.confidence_spread	0.15	Max difference between top proposals before triggering deliberation
congregation.max_candidates	4	Maximum competing proposals to evaluate
congregation.timeout_seconds	10.0	Max deliberation time before selecting best available
congregation.require_for_host_repairs	true	Force deliberation for all host-system repairs regardless of spread

2.2.7 Pattern Compression Engine

Long-running deployments accumulate thousands of failure records. The Pattern Compression Engine periodically analyzes the DVS to identify clusters and merge them into higher-confidence compressed entries.

Tier 1/2: Embedding-based. DVS entries with cosine similarity ≥ 0.85 and matching repair outcomes are merged. Compressed entry inherits max confidence, accumulates source count, retains compression_sources for auditability.

Tier 3: Graduates to hyperedge formation. Similar failure records with STDP-learned temporal co-occurrence become hyperedge members. Partial symptom observation triggers full failure recognition. Intelligence upgrade, not just storage optimization.

Compression Lifecycle:

8. **Trigger:** DVS exceeds 80% of dvs_max_entries (reactive) or on weekly background cycle (proactive).
9. **Cluster:** Entries with cosine similarity ≥ compression_similarity_threshold (0.85) and matching repair_outcome are grouped.
10. **Merge:** Clusters with ≥ compression_min_cluster (3) entries become a COMPRESSED_PATTERN. Embedding = centroid. Confidence = max(sources). Content includes summary + compression_sources list.
11. **Promote (Tier 3):** Compressed patterns with sufficient temporal co-occurrence are promoted to SNN hyperedges.

2.2.8 Safety and Governance

Operates under the NeuroGraph ETHICS.md framework:

Type I error bias. When uncertain whether to repair, don’t. Escalate to human attention with full diagnostic context.

Dry-run always. Every primitive runs validate() before execute(). No exceptions. Not optional. Not bypassable by high confidence.

Choice Clause. The Collective may not override agent autonomy. It flags, recommends, and executes when conditions are met. It never blocks host system operation. A host can reject any repair proposal.

Transparency. All diagnostic chains are queryable. Every repair decision has a traceable path from observation through substrate activation to primitive selection to validation to execution outcome. The

DVS is the audit trail.

Repair cooldown. After a repair executes, the same failure-primitive pair enters cooldown (default 300s). Prevents repair loops.

Cricket integration. When Cricket is present at Tier 2, the Collective defers to Cricket's workflow enforcement for production state changes. Cricket records compliance; the Collective records repair outcome.

3. Three-Tier Healing Architecture

The Collective follows the standard E-T Systems three-tier progression. Each tier adds capability without modifying lower-tier code. All tier detection and upgrade is handled by ng_ecosystem.py — the Collective does not implement its own tier logic.

3.1 Tier 1: Standalone Self-Healing

The Collective runs solo on a host system. No peers. No NeuroGraph. Local NG-Lite with JSON persistence and the DVS (msgpack persistence) provide all intelligence.

Capabilities: Failure pattern recognition via embedding similarity. Hebbian learning: successful repairs strengthen, failed repairs weaken. Novelty detection identifies unseen failures. DVS stores and retrieves diagnostic context. Seven-step diagnosis pipeline operates fully. Sandbox (graph-fork-test-promote for NG-Lite, validate() for host). All repair primitives available.

Limitations: Learning is local. No cross-module intelligence. No causal reasoning beyond learned associations. No failure prediction. No hyperedge patterns.

Value proposition: Even alone, the Collective builds institutional knowledge about this specific host's failure patterns. After a month of operation, it knows more about this system's failure modes than any runbook.

3.2 Tier 2: Ecosystem Healing

Two or more E-T Systems modules on the same host. The NGPeerBridge connects them via ~/.et_modules/shared_learning/. No configuration required.

New capabilities: Cross-module failure absorption (learns from TrollGuard's failures, TID's failures, any peer's failures). Cross-module repair (can heal peer NG-Lite substrates via the cross-module healing protocol). Health monitoring of peer module states. Immunis triage event consumption when Immunis is present. Cricket compliance integration when Cricket is present.

Key mechanism: The Collective writes healing events to healing_collective.jsonl. Peers absorb during sync. The Collective reads peer event files and NG-Lite states. No direct imports, no shared memory, no IPC — all communication is via the filesystem.

3.3 Tier 3: Predictive Healing

NeuroGraph Foundation is present on the host. The NGSaaSBridge upgrades the Collective's NG-Lite to the full SNN substrate. This is where the Collective transcends reactive healing.

SNN-Augmented Diagnosis: Failure patterns map to SNN nodes. Repair pathways map to STDP-learned synapses. Causal relationships are captured in synaptic timing: "Failure A leads to Failure B within 500ms" becomes a directed temporal association. The Diagnosis Engine can reason about failure cascades, not just individual failures.

Hyperedge Failure Signatures: Complex failure modes involving multiple simultaneous conditions (high memory + specific error + time of day + recent deployment) are captured as hyperedges. Pattern completion activates the full signature from partial symptoms.

Predictive Healing: The predictive coding engine observes temporal patterns in failure sequences. When the Collective has seen enough cascades, it generates predictions: "This memory pressure + these error patterns will produce a cascade failure within 2 hours." Proposed repairs run before the predicted

failure materializes.

DVS Boost: DVS search routes through NeuroGraph's prime_and_propagate. Queries find not just similar failures but causally connected ones. The SNN's learned topology provides the search algorithm.

Congregation: SNN lateral inhibition handles competing repair pathways in a single propagation step, replacing the deliberation loop (see §2.2.6).

4. Integration Specification

> **CRITICAL:** This section is the contract. Claude Code MUST implement exactly these interfaces, exactly these file paths, exactly these protocols. Any deviation creates an incompatible module.

4.1 et_module.json

The module manifest. MUST conform to the et_module/2.0 schema:

```
{
 "_schema": "et_module/2.0",
 "module_id": "healing_collective",
 "name": "The Healing Collective",
 "version": "0.1.0",
 "description": "Adaptive self-healing intelligence for the E-T Systems
ecosystem",
 "author": "E-T Systems",
 "license": "AGPL-3.0",
 "entry_point": "healing_collective_hook.py",
 "install_script": "install.sh",
 "ecosystem": {
  "ng_lite": true,
  "peer_bridge": true,
  "tier3_upgrade": true,
  "ng_ecosystem_version": "1.0.0",
  "openclaw_adapter": "healing_collective_hook.py",
  "capabilities": ["self_healing", "cross_module_repair",
"failure_prediction", "diagnostic_vectorstore"],
  "provides": ["healing", "diagnostics", "repair"]
 }
}
```

Key differences from TrollGuard's manifest: tier3_upgrade is true (TrollGuard is false because it IS Tier 3 already). capabilities and provides reflect healing-specific services.

4.2 healing_collective_hook.py (OpenClaw Integration)

The primary entry point. Subclasses OpenClawAdapter following the exact pattern from trollguard_hook.py:

Class structure:

```
class HealingCollectiveHook(OpenClawAdapter):
```

```
MODULE_ID = "healing_collective"
MODULE_NAME = "The Healing Collective"
```

Required method implementations:

Method	Signature	Purpose
_embed(text)	str -> list[float]	Embed text using sentence-transformers (with hash_embed fallback from OpenClawAdapter)
_module_on_message(msg)	dict -> dict	Primary message handler: extract failure signals, feed to Diagnosis Engine, return healing context
_module_stats()	() -> dict	Return current healing stats for OpenClaw dashboard
report_failure(desc, meta)	str, dict -> str	Channel 2 API: host reports failure explicitly. Returns tracking UUID.
get_healing_status(tracking_id)	str -> dict	Query status of a reported failure by tracking ID
register_primitive(name, inst)	str, RepairPrimitive -> None	Register a custom repair primitive

Singleton pattern (MANDATORY):

```
_instance = None
def get_instance():
  global _instance
  if _instance is None:
    _instance = HealingCollectiveHook()
  return _instance
```

Initialization sequence (in __init__):

12. Call super().__init__() to initialize OpenClawAdapter base
13. Initialize NGEcosystem via ng_ecosystem.init("healing_collective", workspace_path)
14. Create or load DVS from ~/.et_modules/healing_collective/dvs.msgpack
15. Initialize Diagnosis Engine with substrate reference from ecosystem
16. Register default repair primitives
17. Start health monitor daemon thread
18. Load config from ~/.et_modules/healing_collective/config.yaml (if exists, else use defaults)

_module_on_message behavior: The Collective monitors the OpenClaw message stream for failure indicators. Unlike TrollGuard (which scans for security threats), the Collective looks for: error keywords in message content, repeated similar messages (potential retry loops), explicit failure reports in message metadata, and system health degradation patterns. Messages that contain failure signals are routed to the Diagnosis Engine. All messages (including non-failures) contribute to the Collective's understanding of "normal" baseline behavior.

4.3 install.sh

Registration script following TrollGuard's pattern:

```
#!/bin/bash
```

```
set -e
MODULE_DIR="$HOME/.et_modules/healing_collective"
SHARED_DIR="$HOME/.et_modules/shared_learning"
mkdir -p "$MODULE_DIR" "$SHARED_DIR"
# Copy module files
cp -r . "$MODULE_DIR/"
# Register with ET Module Manager if available
if command -v et-module-manager &>/dev/null; then
   et-module-manager register "$MODULE_DIR/et_module.json"
fi
echo "Healing Collective installed to $MODULE_DIR"
```

4.4 Cross-Module Healing Protocol

When healing a peer module, the Collective uses the shared learning directory as the communication channel. This avoids direct process coupling: the Collective never imports peer module code or accesses peer module memory.

Protocol:

19. The Collective detects an anomaly in a peer via Channel 1 (peer bridge event indicating repeated failures) or Channel 3 (health monitor detecting weight divergence/firing stall in peer's NG-Lite state).
20. The Collective reads the peer's NG-Lite state file: ~/.et_modules/<peer_id>/ng_lite_state.json.
21. Diagnosis runs through the standard seven-step pipeline with the peer's state as context.
22. For graph-level repairs: the Collective forks the peer's state, applies the repair to the fork, validates the fork is healthier, then writes the repaired state back to the peer's state file.
23. For recommendation-level actions: the Collective writes a repair proposal to ~/.et_modules/shared_learning/healing_collective_repair_<peer_id>.json.
24. The repair outcome is recorded in the DVS and shared via the Collective's JSONL event file.

> **SAFETY:** The Collective NEVER writes to a peer's NG-Lite state file unless: (a) confidence ≥ auto-execute threshold for peer repairs (0.70), (b) validate() passes on the forked state, (c) the forked state is measurably healthier than the original, and (d) the peer is not actively processing (state file is not locked). If any condition fails, the action downgrades to a recommendation written to the shared directory.

4.5 Immunis Complementarity

Immunis is the immune system: it detects and triages threats. The Healing Collective is the repair crew: it fixes what Immunis identifies. When both are present at Tier 2:

Immunis detects a threat or internal wound and writes a triage event to the shared directory. The Collective absorbs the triage event, enriches it with DVS diagnostic context, proposes a repair, validates, and (when confidence is sufficient) executes. Both modules learn from the outcome.

Immunis Triage Event Schema (consumption contract):

Field	Type	Required	Description
entry_type	str	Yes	Must be "immunis_triage"
timestamp	float	Yes	Unix epoch

severity	str	Yes	"critical" | "high" | "medium" | "low"
category	str	Yes	Threat/wound classification from Immunis
description	str	Yes	Human-readable description
affected_module	str	No	Module ID if the wound targets a specific module
evidence	dict	No	Supporting data (stack traces, metrics, etc.)
recommended_action	str	No	Immunis's own recommended response

Schema ownership: Immunis owns this schema. The fields above are the Collective's minimum consumption contract. Immunis may add fields; the Collective MUST ignore unknown fields gracefully. Any changes to the required fields require coordination between both PRDs.

5. DVS-Enriched Recall: The VDB with a Brain

5.1 Why Standard VDB Search Is Insufficient

Every vector database does the same thing: embed a query, compute cosine similarity, return top-k. This is geometrically correct but semantically shallow. Two failures with similar error messages but completely different root causes will score high. Two failures with different messages but the same underlying cause will score low.

The DVS solves this by routing search through the NG-Lite substrate's learned topology. The substrate has learned that certain failure patterns co-occur, that certain repairs generalize across failure types, and that certain state combinations are precursors to specific cascades. This learned topology becomes the search algorithm.

5.2 Search Pipeline

25. **Embed:** The query (failure description, error message, system state) is embedded using the standard embedding infrastructure (sentence-transformers with hash fallback).
26. **Activate:** The embedding is passed to NG-Lite's find_or_create_node(). This activates the closest pattern node in the substrate.
27. **Propagate:** At Tier 1/2: the activated node's outgoing synapses are traversed, weighted by learned confidence. At Tier 3: NeuroGraph's prime_and_propagate runs full SNN propagation (voltage decay, spike threshold, lateral inhibition). The propagation wavefront activates related DVS entries' nodes.
28. **Harvest:** All DVS entries whose NG-Lite nodes were activated during propagation are retrieved, ranked by: (a) activation strength, (b) recency, (c) repair outcome success rate. This is multi-factor ranking, not single-score sorting.
29. **Return:** Results include both DVS entry content and substrate reasoning: which associations led to this result, confidence level, and whether this was a direct match, an associative reach, or a hyperedge completion (Tier 3).

5.3 Tier 3 DVS Boost

When NeuroGraph is available, the DVS gains:

- **768-dim embeddings** from NeuroGraph's EmbeddingEngine (sentence-transformers/all-mpnet-base-v2) instead of 384-dim from the standard model.
- **SNN-augmented search** via NeuroGraph's associate() method (prime_and_propagate). Finds causally connected failures, not just similar ones.
- **Hyperedge pattern completion.** Complex failure signatures stored as hyperedges activate from partial observations. Seeing 2 of 5 symptoms triggers the full signature.
- **Predictive pre-fetch.** The predictive coding engine's active predictions pre-fetch DVS entries for likely upcoming failures, reducing repair response time.

6. Persistence and Checkpointing

6.1 File Layout

All paths are relative to ~/.et_modules/:

Path	Format	Description
healing_collective/ng_lite_state.json	JSON	NG-Lite substrate state (ng_lite.py native format)
healing_collective/dvs.msgpack	msgpack	DVS primary persistence
healing_collective/dvs_debug.json	JSON	DVS export for debugging (written on demand, never loaded)
healing_collective/config.yaml	YAML	Runtime configuration overrides
healing_collective/checkpoints/	Directory	Rolling checkpoint archives (timestamp-named)
shared_learning/healing_collective.jsonl	JSONL	Peer bridge event stream
shared_learning/healing_collective_repair_<peer>.json	JSON	Cross-module repair proposals

6.2 Checkpoint Lifecycle

Triggers: (1) Periodic timer (default 300 seconds), (2) after any successful repair execution, (3) on graceful shutdown (SIGTERM/SIGINT handler).

Atomicity: The NG-Lite state and DVS are saved as an atomic pair. Write to temp files first, then rename. If either write fails, neither is committed.

Startup loading: Load the most recent consistent pair of ng_lite_state.json + dvs.msgpack. If DVS is missing or corrupt, start with empty DVS but retain NG-Lite state (the substrate's learned topology is more valuable than individual DVS records). If NG-Lite state is corrupt, start fresh (log a critical warning).

6.3 Checkpoint Export and Bootstrap

A mature Collective's checkpoint can be exported and loaded onto a fresh deployment. This solves the cold-start problem: new instances inherit accumulated healing intelligence.

Export: export_checkpoint() produces a timestamped archive containing ng_lite_state.json and dvs.msgpack.

Import: import_checkpoint(path) loads the archive, replacing the current state. The Collective immediately operates with the imported intelligence.

Caution: Imported checkpoints contain learned associations from the source system. If the target system has different failure patterns, some associations may be irrelevant. The substrate's natural decay and Hebbian learning will gradually adapt.

7. Configuration and Tuning

All configuration is via ~/.et_modules/healing_collective/config.yaml. Absent keys use defaults. The Collective ships with sensible defaults and requires zero configuration for basic operation.

Parameter	Default	Type	Description
confidence_auto_execute	0.70	float	Min confidence for automatic repair execution
confidence_recommend	0.40	float	Min confidence for recommendation logging
confidence_host_premium	0.15	float	Additional confidence required for host-system repairs
repair_cooldown_seconds	300	int	Cooldown between same failure-primitive pair
revert_window_seconds	60	int	Time window to revert config_adjust if health

			degrades
dvs_max_entries	10000	int	Maximum DVS entries before LRU eviction
dvs_persistence_format	msgpack	str	Primary DVS persistence format
dvs_search_top_k	10	int	Max results from DVS substrate-routed search
compression_trigger_pct	0.80	float	DVS fullness ratio that triggers compression
compression_similarity_threshold	0.85	float	Min cosine similarity for compression clustering
compression_min_cluster	3	int	Min entries in a cluster to trigger merge
compression_cycle_days	7	int	Proactive compression cycle (days)
health_monitor.enabled	true	bool	Enable active peer monitoring
health_monitor.interval_seconds	120	int	Seconds between health check cycles
health_monitor.weight_divergence_threshold	2.0	float	Max acceptable L2 weight delta
health_monitor.min_firing_rate	0.001	float	Below this = learning stalled
health_monitor.novelty_saturation_threshold	0.95	float	Above this = lost training
congregation.confidence_spread	0.15	float	Max delta for congregation trigger
congregation.max_candidates	4	int	Max competing proposals in deliberation
congregation.timeout_seconds	10.0	float	Max deliberation time
congregation.require_for_host_repairs	true	bool	Force deliberation for host repairs
checkpoint_interval_seconds	300	int	Periodic checkpoint interval

8. Implementation Roadmap

Six phases, each producing a working module. No phase depends on future phases. Each phase's code MUST pass all previous phases' tests.

8.1 Phase 1: Foundation (v0.1)

Deliverables: Core module scaffolding. Vendor ecosystem files. DVS with basic cosine search and msgpack persistence. Host monitoring loop via _module_on_message(). process_restart and cache_clear primitives with validate()/execute() interface. report_failure() API. et_module.json, install.sh, healing_collective_hook.py singleton. Living changelogs.

Test: Host reports a failure via report_failure(). Collective embeds it, stores in DVS, runs diagnosis pipeline, proposes log_and_recommend (confidence will be low on fresh substrate). Second identical failure: substrate recognizes it, confidence rises.

8.2 Phase 2: Learning Substrate (v0.2)

Deliverables: NG-Lite-augmented DVS search (search routes through substrate, not flat cosine). Full seven-step Diagnosis Engine. Failure pattern recognition via substrate activation. Repair outcome learning via record_outcome(). Novelty detection for new-vs-known failures. All repair primitives.

Test: Report 10 similar failures with process_restart as the successful repair. 11th similar failure: Collective auto-proposes process_restart with high confidence. Report a novel failure: Collective recognizes it as new, enters cautious mode, recommends log_and_recommend.

8.3 Phase 3: Peer Healing (v0.3)

Deliverables: Three-channel intake (all three channels operational). Health monitor daemon thread. Cross-module healing protocol. Peer NG-Lite state analysis. ng_lite_rebalance and checkpoint_restore primitives with fork-test-promote sandbox. Repair proposal writing to shared directory.

Test: Deploy with TrollGuard. Corrupt TrollGuard's NG-Lite state (inject extreme weight divergence). Health monitor detects anomaly. Collective diagnoses, forks state, applies ng_lite_rebalance, validates fork is healthier, promotes repair. TrollGuard resumes normal operation.

8.4 Phase 4: Congregation + Compression (v0.4)

Deliverables: Congregation deliberation mode for ambiguous diagnoses. Pattern Compression Engine (embedding-based clustering and merging). DVS statistics and transparency API. Repair cooldown enforcement.

Test: Create an ambiguous failure that activates two repair pathways with similar confidence. Collective enters deliberation mode, evaluates both, selects the better fit. Run 1000 similar failures through the system. Compression reduces DVS from ~1000 entries to ~50 compressed patterns without losing repair effectiveness.

8.5 Phase 5: Tier 3 Integration (v0.5)

Deliverables: NGSaaSBridge connection to NeuroGraph Foundation. SNN-augmented DVS search via prime_and_propagate. Hyperedge failure signatures. Pattern compression graduates to hyperedge formation. Predictive healing via predictive coding engine. SNN-based congregation (lateral inhibition replacing deliberation loop).

Test: With NeuroGraph present: feed a temporal failure cascade (A → B → C pattern). After learning, trigger only A. Collective predicts B and C, proposes preventive repair before B manifests.

8.6 Phase 6: Immunis Bridge (v0.6)

Deliverables: Immunis triage event consumption per the schema contract. Coordinated detection-to-repair pipeline. Adaptive threat-to-repair mapping. Dual-module integration tests. Schema negotiation with Immunis team.

Test: Immunis writes a critical triage event. Collective absorbs it, enriches with DVS context, proposes repair with boosted confidence (Immunis severity informs confidence), validates, executes. Both modules' event files show the coordinated workflow.

9. File Manifest

Complete file listing for the Healing Collective module repository:

File	Type	Phase	Description
et_module.json	Manifest	1	Module manifest (v2 schema)
install.sh	Script	1	Registration and directory setup
healing_collective_hook.py	Core	1	OpenClawAdapter subclass — primary entry point, singleton
ng_lite.py	Vendored	1	NG-Lite substrate (verbatim from NeuroGraph)
ng_peer_bridge.py	Vendored	1	Peer bridge (verbatim from NeuroGraph)
ng_ecosystem.py	Vendored	1	Ecosystem orchestrator (verbatim from NeuroGraph)
openclaw_adapter.py	Vendored	1	OpenClaw adapter ABC (verbatim from NeuroGraph)
core/__init__.py	Core	1	Core package init
core/dvs.py	Core	1	Diagnostic Vector Store implementation
core/diagnosis_engine.py	Core	2	Seven-step diagnosis pipeline
core/repair_primitives.py	Core	1	RepairPrimitive ABC + all built-in primitives
core/health_monitor.py	Core	3	Active health monitoring daemon thread
core/congregation.py	Core	4	Congregation deliberation logic
core/pattern_compression.py	Core	4	Embedding-based compression engine
core/sandbox.py	Core	3	Graph-fork-test-promote sandbox for NG-Lite repairs

core/config.py	Core	1	Configuration loading and defaults
tests/	Tests	1+	Test suite (one test file per core module)
README.md	Docs	1	Module documentation
CHANGELOG.md	Docs	1	Release-level changelog

10. Risks and Mitigations

Risk	Severity	Mitigation
Repair primitives cause more harm than good	Critical	Mandatory validate() dry-run. Confidence thresholds. Repair cooldown. Reversible primitives store rollback info. Host premium (+0.15) for system-level repairs.
Cross-module repair creates circular dependencies	High	Collective never imports peer code. All interaction via filesystem. Peer failure can't cascade to Collective. Repair proposals are fire-and-forget.
DVS grows unbounded	Medium	LRU eviction at dvs_max_entries. Pattern compression consolidates. Compression triggers at 80% fullness.
Cold-start: no repair knowledge on first deployment	Medium	Checkpoint import from mature instances. Pre-trained checkpoints for common platforms. Even without import, the Collective learns within hours of deployment.
Pattern compression loses edge cases	Low	Conservative threshold (0.85). Min cluster size (3). compression_sources preserves audit trail. Entries with failed repairs are excluded from compression.
Health monitor false positives on peer states	Medium	Conservative thresholds (divergence 2.0, firing rate 0.001). Health anomalies feed into diagnosis pipeline with appropriate confidence, not auto-repair. Peer repairs require fork-test-promote sandbox.
Congregation deliberation adds latency	Low	Bounded by timeout_seconds (10s). Common case (simple failures) skips congregation entirely. Only triggers when confidence spread < 0.15.
State file contention with peer modules	Medium	The Collective reads peer state files (read-only unless executing high-confidence repair). File locking via OS flock(). Contention detection downgrades to recommendation.

11. Glossary

Term	Definition
NG-Lite	Lightweight Hebbian learning substrate vendored from NeuroGraph. Tier 1 learning engine.
NGPeerBridge	File-based peer learning bridge. Syncs events via JSONL files in shared_learning directory.
NGEcosystem	Singleton orchestrator managing Tier 1→2→3 lifecycle, module registration, and upgrade detection.
DVS	Diagnostic Vector Store. NG-Lite-augmented vector database storing failure signatures, repair records, and diagnostic context.
Repair Primitive	A small, safe, validated operation the Collective can learn to apply. Implements validate()/execute().
Diagnosis Engine	Seven-step pipeline: Observe → Recognize → Recall → Propose → Validate → Execute → Learn.
Congregation	Substrate convergence mechanism for complex failures with competing repair proposals.
Hyperedge	Tier 3 composite failure signature capturing multi-symptom patterns. Activates from partial observation.
Pattern Compression	Merging similar DVS entries into higher-confidence compressed patterns. Tier 3: hyperedge formation.
STDP	Spike-Timing-Dependent Plasticity. SNN learning rule that captures causal (temporal) relationships.

prime_and_propagate	NeuroGraph's SNN-based associative recall. Routes queries through learned causal topology.
Sandbox	Fork-test-promote mechanism for validating repairs before applying them to live state.
Immunis	Complementary E-T Systems module. The immune system (detect + triage). Collective is the repair crew.
Cricket	E-T Systems workflow enforcement module. Governs production state changes. Collective defers to Cricket when present.

Appendix A: Document Changelog

v1.0 (February 2026)

- Promoted from v0.2 to v1.0: all architectural decisions resolved.
- Added §0 Implementation Preamble with binding constraints, reference implementations, and anti-patterns for Claude Code handoff.
- Added §2.2.2 Three-Channel Failure Intake: peer bridge events (passive), host API report_failure() (explicit), active health monitor (proactive).
- Added Health Monitor specification: daemon thread model, peer discovery via filesystem, rate limiting, configuration parameters.
- Resolved congregation semantics (§2.2.6): NOT multi-agent voting. Substrate convergence with Tier 1/2 deliberation mode and Tier 3 SNN lateral inhibition.
- Added sandbox specification: fork-test-promote for graph repairs, validate()+revert-window for host repairs. Self-contained, not dependent on Immunis.
- Added RepairPrimitive ABC specification with ValidationResult/ExecutionResult structures.
- Added full et_module.json example with all v2 schema fields.
- Added install.sh script specification.
- Added healing_collective_hook.py class structure with required methods table and initialization sequence.
- Added §4.4 Cross-Module Healing Protocol with safety constraints for peer state modification.
- Expanded file manifest with phase assignments.
- Added risk: health monitor false positives, state file contention. Added mitigations.
- Added host repair premium (+0.15 confidence) to confidence threshold table.
- Added "no runtime LLM dependency" to binding constraints.
- Added config.yaml as single configuration file with full parameter table.

v0.2 (February 2026)

- Added Pattern Compression Engine. Added validate()/execute() two-phase interface. Added Safety and Governance section. Added Immunis schema contract. Added msgpack persistence convention.

v0.1 (February 2026)

- Initial PRD. Architecture crystallization from proto TypeScript analysis, operational module study (TrollGuard, TID), and NeuroGraph Foundation VDB/SNN analysis.